



OBSERVATORIO
ASTRONÓMICO
NACIONAL



Ecuaciones Diferenciales Ordinarias

Otros Métodos

Dr. rer. nat. Jose Ivan Campos Rozo

Astronomical Institute of the Czech Academy of Sciences
Observatorio Astrónomico Nacional de Colombia - UNAL

May 9, 2026

CONTENIDO

1 Integradores Simpléticos

- Integrador LeapFrog
- Integrador Verlet

Integrador LeapFrog

LeapFrog and Verlet integrators are introduced to solve ODE systems. (A. Garcia. Numerical Methods for Physics. (1999). Chapter 3)

Consideremos el siguiente sistema de EDO,

$$\frac{d^2x}{dt^2} = a(x), \quad (1)$$

Integrador LeapFrog

LeapFrog and Verlet integrators are introduced to solve ODE systems. (A. Garcia. Numerical Methods for Physics. (1999). Chapter 3)

Consideremos el siguiente sistema de EDO,

$$\frac{d^2x}{dt^2} = a(x), \quad (1)$$

que puede escribirse como el sistema diferencial de primer orden

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = a(x). \end{cases} \quad (2)$$

Integrador LeapFrog

LeapFrog and Verlet integrators are introduced to solve ODE systems. (A. Garcia. Numerical Methods for Physics. (1999). Chapter 3)

Consideremos el siguiente sistema de EDO,

$$\frac{d^2x}{dt^2} = a(x), \quad (1)$$

que puede escribirse como el sistema diferencial de primer orden

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = a(x). \end{cases} \quad (2)$$

Nótese la dependencia de la función $a = a(x(t))$ del lado derecho de las ecuaciones.

La discretización de la derivada temporal (utilizando diferencias centradas) en la segunda ecuación da como resultado:

$$\frac{v(t + \Delta t) - v(t - \Delta t)}{2\Delta t} + \mathcal{O}(\Delta t^2) = a(x). \quad (3)$$

La discretización de la derivada temporal (utilizando diferencias centradas) en la segunda ecuación da como resultado:

$$\frac{v(t + \Delta t) - v(t - \Delta t)}{2\Delta t} + \mathcal{O}(\Delta t^2) = a(x). \quad (3)$$

En la primera ecuación diferencial también se utilizará una discretización centrada de la derivada, pero en este caso estará centrada en el punto $t + \Delta t$ (esto significa que los puntos $t + 2\Delta t$ y t en la diferencia finita). Con esto se obtiene

$$\frac{x(t + 2\Delta t) - x(t)}{2\Delta t} + \mathcal{O}(\Delta t^2) = v(t + \Delta t). \quad (4)$$

Estas dos ecuaciones en diferencias se escriben iterativamente como:

$$\begin{cases} \frac{x_{n+2}-x_n}{2\Delta t} + \mathcal{O}(\Delta t^2) &= v_{n+1} \\ \frac{v_{n+1}-v_{n-1}}{2\Delta t} + \mathcal{O}(\Delta t^2) &= a_n. \end{cases} \quad (5)$$

Estas dos ecuaciones en diferencias se escriben iterativamente como:

$$\begin{cases} \frac{x_{n+2}-x_n}{2\Delta t} + \mathcal{O}(\Delta t^2) &= v_{n+1} \\ \frac{v_{n+1}-v_{n-1}}{2\Delta t} + \mathcal{O}(\Delta t^2) &= a_n. \end{cases} \quad (5)$$

Reorganizando estas ecuaciones, obtenemos el método de iteración conocido como **LeapFrog**,

$$\begin{cases} v_{n+1} &= v_{n-1} + 2a_n\Delta t + \mathcal{O}(\Delta t^3) \\ x_{n+2} &= x_n + 2v_{n+1}\Delta t + \mathcal{O}(\Delta t^3) \end{cases} \quad (6)$$

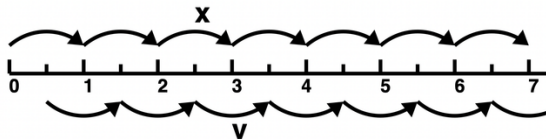
Este es un método de orden $\mathcal{O}(\Delta t^3)$ y su nombre proviene del hecho de que la solución se construye con pasos de tamaño $2\Delta t$, con la función de posición x avanzando en valores pares, $x_0, x_2, x_4, \dots$, mientras que la función de velocidad v avanza en valores impares $v_1, v_3, v_5, \dots$.

Este es un método de orden $\mathcal{O}(\Delta t^3)$ y su nombre proviene del hecho de que la solución se construye con pasos de tamaño $2\Delta t$, con la función de posición x avanzando en valores pares, $x_0, x_2, x_4, \dots$, mientras que la función de velocidad v avanza en valores impares $v_1, v_3, v_5, \dots$.

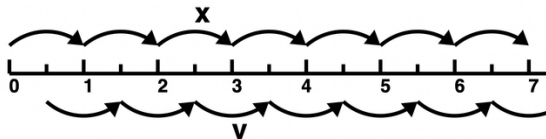
En algunas referencias, las ecuaciones del método LeapFrog se escriben de la siguiente forma (equivalente al método *Euler-Cromer*):

$$\begin{cases} v_{n+\frac{1}{2}} &= v_{n-\frac{1}{2}} + a_n \Delta t + \mathcal{O}(\Delta t^3) \\ x_{n+1} &= x_n + v_{n+\frac{1}{2}} \Delta t + \mathcal{O}(\Delta t^3) \end{cases} \quad (7)$$

El origen del nombre de este método está en la dependencia de la función $a = a(x)$, lo que implica que su valor debe evaluarse en un instante de tiempo centrado entre los valores antiguo y nuevo de la función v .



El origen del nombre de este método está en la dependencia de la función $a = a(x)$, lo que implica que su valor debe evaluarse en un instante de tiempo centrado entre los valores antiguo y nuevo de la función v .



El método LeapFrog es computacionalmente más eficiente que el método Predictor-Corrector y requiere menos almacenamiento de datos. Al ser uno de los algoritmos **simplecticos** (es decir, que preserva el área en el espacio de fase), se respeta la conservación de la energía, incluso con pasos de tiempo grandes.

Iniciación del método LeapFrog

El método Leap-Frog no se inicia automáticamente porque, además de los valores iniciales (x_0, v_0) , es necesario conocer el valor v_{-1} . Una de las posibilidades para iniciar el algoritmo es realizar un paso de Euler hacia atrás para obtener la información requerida,

$$v_{-1} = v_0 - a_0 \Delta t,$$

Iniciación del método LeapFrog

El método Leap-Frog no se inicia automáticamente porque, además de los valores iniciales (x_0, v_0) , es necesario conocer el valor v_{-1} . Una de las posibilidades para iniciar el algoritmo es realizar un paso de Euler hacia atrás para obtener la información requerida,

$$v_{-1} = v_0 - a_0 \Delta t,$$

o medio paso atrás si las ecuaciones están escritas en la segunda forma presentada,

$$v_{-\frac{1}{2}} = v_0 - \frac{1}{2} a_0 \Delta t.$$

Iniciación del método LeapFrog

El método Leap-Frog no se inicia automáticamente porque, además de los valores iniciales (x_0, v_0) , es necesario conocer el valor v_{-1} . Una de las posibilidades para iniciar el algoritmo es realizar un paso de Euler hacia atrás para obtener la información requerida,

$$v_{-1} = v_0 - a_0 \Delta t,$$

o medio paso atrás si las ecuaciones están escritas en la segunda forma presentada,

$$v_{-\frac{1}{2}} = v_0 - \frac{1}{2} a_0 \Delta t.$$

Se puede demostrar que el método LeapFrog es mucho más rápido y estable que el método de Euler.

Ejemplo

```
import numpy as np
G = 4.*np.pi**2  # Newtonian Gravitational Constant
M = 1.
def ODE(q0):
    '''
    -----
    ODEs system for the motion of test body using cartesian
    coordinates in the orbital plane.
    -----
    Arguments:
    q0: NumPy array with the coordinates defined as
    q0[0] = x: coordinate x
    q0[1] = y: coordinate y
    -----
    Returns:
    a = NumPy array with the components of the acceleration
    '''
    r2 = q0[0]**2 + q0[1]**2
    a = - G*M*q0[0:2]/r2**(3/2)
    return a
```

Condición Inicial

Para ilustrar el uso del método LeapFrog, utilizaremos la condición inicial empleada para sondear los métodos Euler y RK4,

- La posición inicial viene dada por $x_0 = 1$ au y $y_0 = 0$ au.
- La velocidad inicial es $v_0^x = 0$ au/año y $v_0^y = 3$ au/año.

Consideraremos un dominio de integración para la coordenada temporal desde $t_0 = 0$ hasta $t_f = 20$ años con un millón de pasos.

Condición Inicial

Para ilustrar el uso del método LeapFrog, utilizaremos la condición inicial empleada para sondear los métodos Euler y RK4,

- La posición inicial viene dada por $x_0 = 1$ au y $y_0 = 0$ au.
- La velocidad inicial es $v_0^x = 0$ au/año y $v_0^y = 3$ au/año.

Consideraremos un dominio de integración para la coordenada temporal desde $t_0 = 0$ hasta $t_f = 20$ años con un millón de pasos.

Nota: El método Leap-Frog no se autoinicia porque necesitamos conocer las condiciones iniciales, (x_0, y_0) y (v_0^x, v_0^y) , junto con algunos valores previos (v_{-1}^x, v_{-1}^y) . Para obtener estos valores previos, podemos usar un paso de Euler hacia atrás,

$$v_{-1}^x = v_0^x - h a^x(x_0, y_0) \quad (8)$$

$$v_{-1}^y = v_0^y - h a^y(x_0, y_0). \quad (9)$$

Ejemplo LeapFrog

```
def LeapFrog(h, q0):  
    '''  
    LeapFrog method for solving a ODEs system.  
    Arguments: h: stepsize for the iteration  
               t0: independent parameter initial value  
               q0: numpy array with the initial values of the  
                   functions in the ODEs system  
                   q[0] : x  
                   q[1] : y  
                   q[2] : dx/dt  
                   q[3] : dy/dt  
    '''  
    ax, ay = ODE(q0)  
    q1 = np.zeros(q0.shape)  
    q1[2] = q0[2] + 2*h*ax  
    q1[3] = q0[3] + 2*h*ay  
    q1[0] = q0[0] + 2*h*q1[2]  
    q1[1] = q0[1] + 2*h*q1[3]  
    return q1
```

```
t_0 = 0.  #(in years)
t_f = 20.  #(in years)
n = 500000  #Number of steps in the grid
h = (t_f - t_0)/n  #Constant stepsize

t = np.linspace(t_0, t_f, n)  # Time information
Q = np.zeros([n,4])  # Motion information

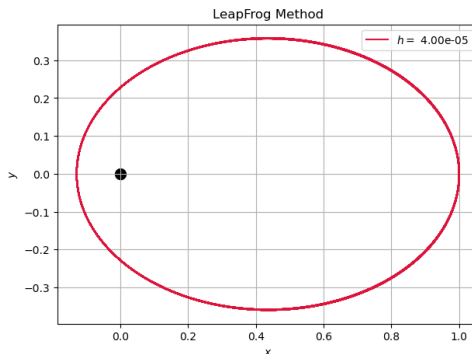
 # Initial Conditions
Q[0,0] = 1.  # initial x
Q[0,1] = 0.  # initial y
Q[0,2] = 0.  # initial vx
Q[0,3] = 3.  # initial vy

 # Half step backwards to initialize the velocity
ax0, ay0 = ODE(Q[0])
Q[0,2] = Q[0,2] - h*ax0
Q[0,3] = Q[0,3] - h*ay0

 # Main loops for solving the problem
for i in range(1,n):
    Q[i,:] = LeapFrog(h, Q[i-1,:])
```

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7,5))
ax.plot(Q[:,0], Q[:,1], color='crimson', label=f'$h=${h:.2e}$')
ax.scatter(0,0, color='black', s=100)
ax.set_title('LeapFrog Method')
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$y$')
ax.legend()
ax.grid()
plt.show()
```

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7,5))
ax.plot(Q[:,0], Q[:,1], color='crimson', label=f'$h=${h:.2e}$')
ax.scatter(0,0, color='black', s=100)
ax.set_title('LeapFrog Method')
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$y$')
ax.legend()
ax.grid()
plt.show()
```



Integrador Verlet

Consideremos la ecuación diferencial ordinaria de segundo orden.

$$\frac{d^2x}{dt^2} = a(x) \quad (10)$$

Integrador Verlet

Consideremos la ecuación diferencial ordinaria de segundo orden.

$$\frac{d^2x}{dt^2} = a(x) \quad (10)$$

escrito en la forma

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{d^2x}{dt^2} = \frac{dv}{dt} = a(x(t)). \end{cases} \quad (11)$$

Integrador Verlet

Consideremos la ecuación diferencial ordinaria de segundo orden.

$$\frac{d^2 x}{dt^2} = a(x) \quad (10)$$

escrito en la forma

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{d^2 x}{dt^2} = \frac{dv}{dt} = a(x(t)). \end{cases} \quad (11)$$

Utilizaremos diferencias centradas para calcular la primera y la segunda derivada y así obtener las ecuaciones iterativas.

$$\begin{cases} \frac{x_{n+1} - x_{n-1}}{2h} + \mathcal{O}(h^2) & = v_n \\ \frac{x_{n+1} - 2x_n + x_{n-1}}{h^2} + \mathcal{O}(h^2) & = a(x_n). \end{cases} \quad (12)$$

Reescribiendo esta ecuación, obtenemos

$$\begin{cases} v_n &= \frac{x_{n+1}-x_{n-1}}{2h} + \mathcal{O}(h^2) \\ x_{n+1} &= 2x_n - x_{n-1} + h^2 a(x_n) + \mathcal{O}(h^4). \end{cases} \quad (13)$$

Reescribiendo esta ecuación, obtenemos

$$\begin{cases} v_n &= \frac{x_{n+1}-x_{n-1}}{2h} + \mathcal{O}(h^2) \\ x_{n+1} &= 2x_n - x_{n-1} + h^2 a(x_n) + \mathcal{O}(h^4). \end{cases} \quad (13)$$

Este método se conoce como el método de **Verlet**. Cabe destacar que este método no se autoinicializa (como el método LeapFrog), ya que necesitamos el valor de x_n y también el de x_{n-1} para obtener x_{n+1} . Asimismo, es importante señalar que la evolución de la posición de la partícula puede obtenerse independientemente de su velocidad.

Ejemplo Verlet

```
def Verlet(h, q0, s0):
```

```
    '''
```

*Verlet method for solving a ODEs. It needs two initial points.
Arguments:*

h: stepsize for the iteration

*q0: numpy array with the values of the coords in the n-step
 q0[0] : x(n); q0[1] : y(n)*

*s0: numpy array with the values of the coords in the (n-1)-step
 s0[0] : x(n-1); s0[1] : y(n-1)*

Returns: Right hand side of the ODEs system as q1:

q1[0] : x(n+1); q1[1] : y(n+1); q1[2] : vx(n); q1[3] : vy(n)
 '''

```
    ax, ay = ODE(q0)
```

```
    q1 = np.zeros(q0.shape)
```

```
    q1[0] = 2*q0[0] - s0[0] + ax*h**2
```

```
    q1[1] = 2*q0[1] - s0[1] + ay*h**2
```

```
    q1[2] = (q1[0] - s0[0])/(2*h)
```

```
    q1[3] = (q1[1] - s0[1])/(2*h)
```

```
    return q1
```

```
t_0 = 0. #(in years)
t_f = 20. #(in years)
n = 1000000 #Number of steps in the grid
h = (t_f - t_0)/n #Constant stepsize
t = np.linspace(t_0, t_f, n) # Time information
Q = np.zeros([n,4]) # Motion information

# Initial Conditions
Q[0,0] = 1. # initial x
Q[0,1] = 0. # initial y
Q[0,2] = 0. # initial vx
Q[0,3] = 3. # initial vy

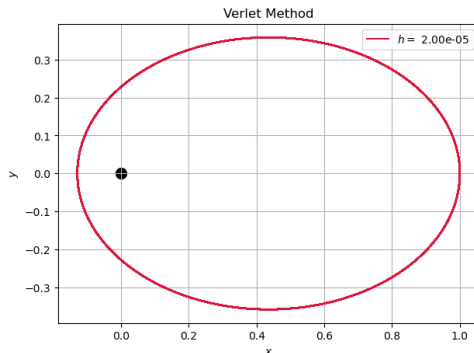
S = np.zeros(2)
ax0, ay0 = ODE(Q[0])
S[0] = Q[0,0] - h*Q[0,2] - 0.5*ax0*h**2
S[1] = Q[0,1] - h*Q[0,3] - 0.5*ay0*h**2
Q[1,:] = Verlet(h, Q[0,:], S)

for i in range(2,n):
    Q[i,:] = Verlet(h, Q[i-1,:], Q[i-2,0:2])
```

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7,5))
ax.plot(Q[:,0], Q[:,1], color='crimson', label=f'$h=${h:.2e}$')
ax.scatter(0,0, color='black', s=100)
ax.set_title('Verlet Method')
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$y$')
ax.grid()
ax.legend()
plt.show()
```



```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7,5))
ax.plot(Q[:,0], Q[:,1], color='crimson', label=f'$h=${h:.2e}$')
ax.scatter(0,0, color='black', s=100)
ax.set_title('Verlet Method')
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$y$')
ax.grid()
ax.legend()
plt.show()
```



Otros Métodos

- Método Bulirsch-Stoer
- Algoritmo Barnes-Hut

Otros Métodos

- Método Bulirsch-Stoer
- Algoritmo Barnes-Hut

Python Solvers:

- SOLVE_IVP
- RK23
- RK45
- DOP853
- ODEINT (API antigua)

Continuará

Dr. rer. nat. Jose Ivan Campos Rozo
jicamposr@unal.edu.co